

Week 7: Visualisation

MY472: Data for Data Scientists

https://lse-my472.github.io/

Autumn Term 2026

Ryan Hübert

Associate Professor of Methodology

Slides last updated: 10 November 2026

The importance of visualisation

Kansas Election Results

Gov. Holscher (D) +0.7 >95% in | **Senate** Hamilton (D) ✓ | **Amendment 1 – Require Citizenship to Vote** Yes ✓

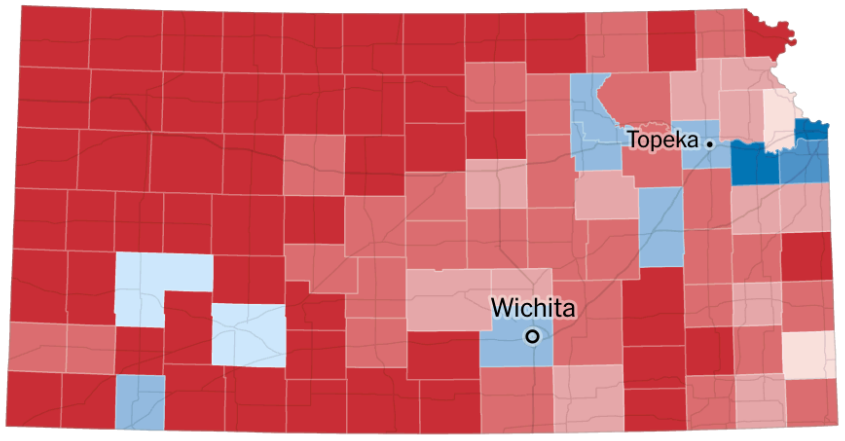
U.S. Senate ›

Senator Roger Marshall, a first-term Republican, is trying to survive an unexpectedly competitive race in deep-red Kansas. He faces Adam Hamilton, a centrist Democrat who has sought to corral farmers' anger at economic conditions under President Trump.

Latest results from Nov. 7 — >95% OF VOTES IN — AP

Candidate	Party	Votes	Pct.
Adam Hamilton ✓	Democrat	561,143	**51.0%**
Roger Marshall*	Republican	505,558	**46.0**
David C. Graham	Libertarian	33,195	**3.0**
Total reported		1,099,896	

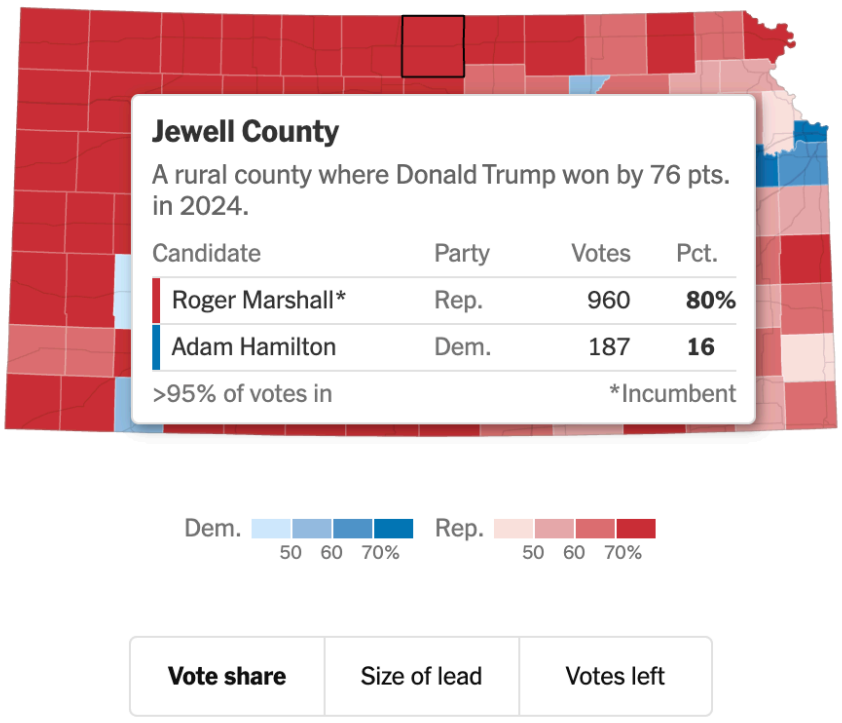
*Incumbent

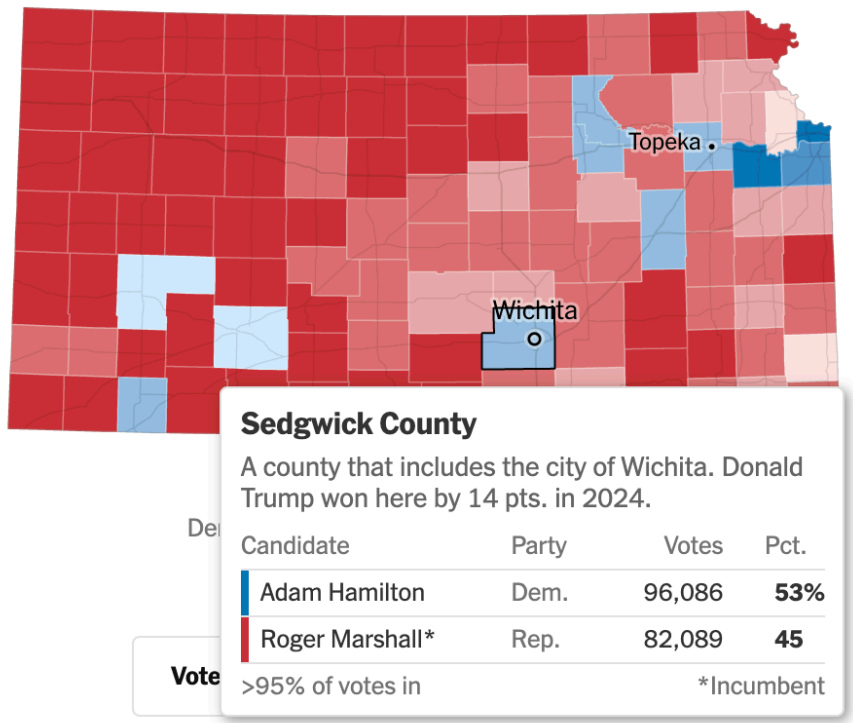


Vote share | Size of lead | Votes left

Source: https://www.nytimes.com/interactive/2026/11/03/us/elections/results-kansas.html

The importance of visualisation





Source: https://www.nytimes.com/interactive/2026/11/03/us/elections/results-kansas.html

Outline

1. Some principles of data visualisation
2. Grammar of graphics and `plotnine`
3. Spatial data

Reducing complexity to enable *learning* from data

A tabular dataset is a complex object—lots of observations (rows), lots of information about those observations (columns)

A typical human can't draw meaning from a raw tabular dataset

The point of data analysis: reduce complexity, enable learning

To learn, you need to **summarise**, e.g.:

- Calculate means/medians/counts/etc. of each variable
- Calculate correlations between multiple variables
- Make plots of distributions ("shapes") of variables

Visual communication of data: **visualisation**

- *Lots* of discretion, need to do this well; could fill a course!

Visualisation and data

So far: how data is stored, read, cleaned and combined

Today: how data is turned into a figure

Making a figure is a series of steps applied to data

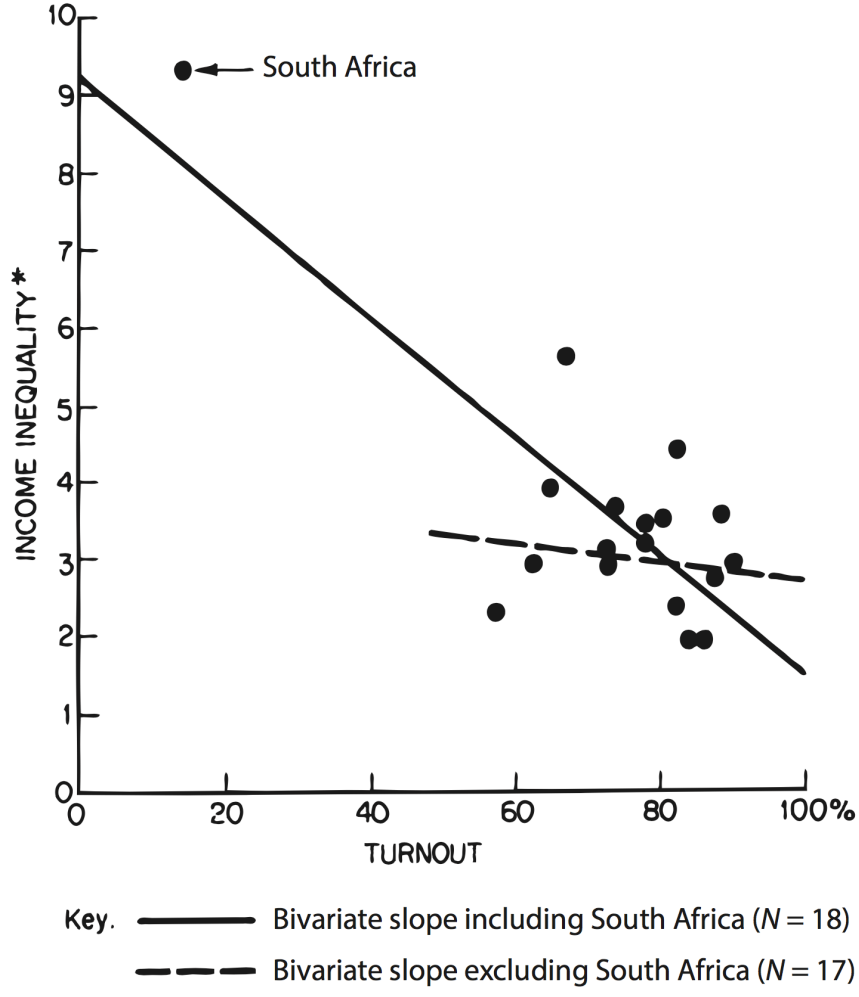
- Each value becomes a position, a length, a colour or a shape
- Each step involves a choice

Examples of these choices:

- Which observations to include
- Which statistic to plot
- What an area on the page stands for

The reader sees the figure, but not the data or the choices

Why you should look at data (from Healy 2019)

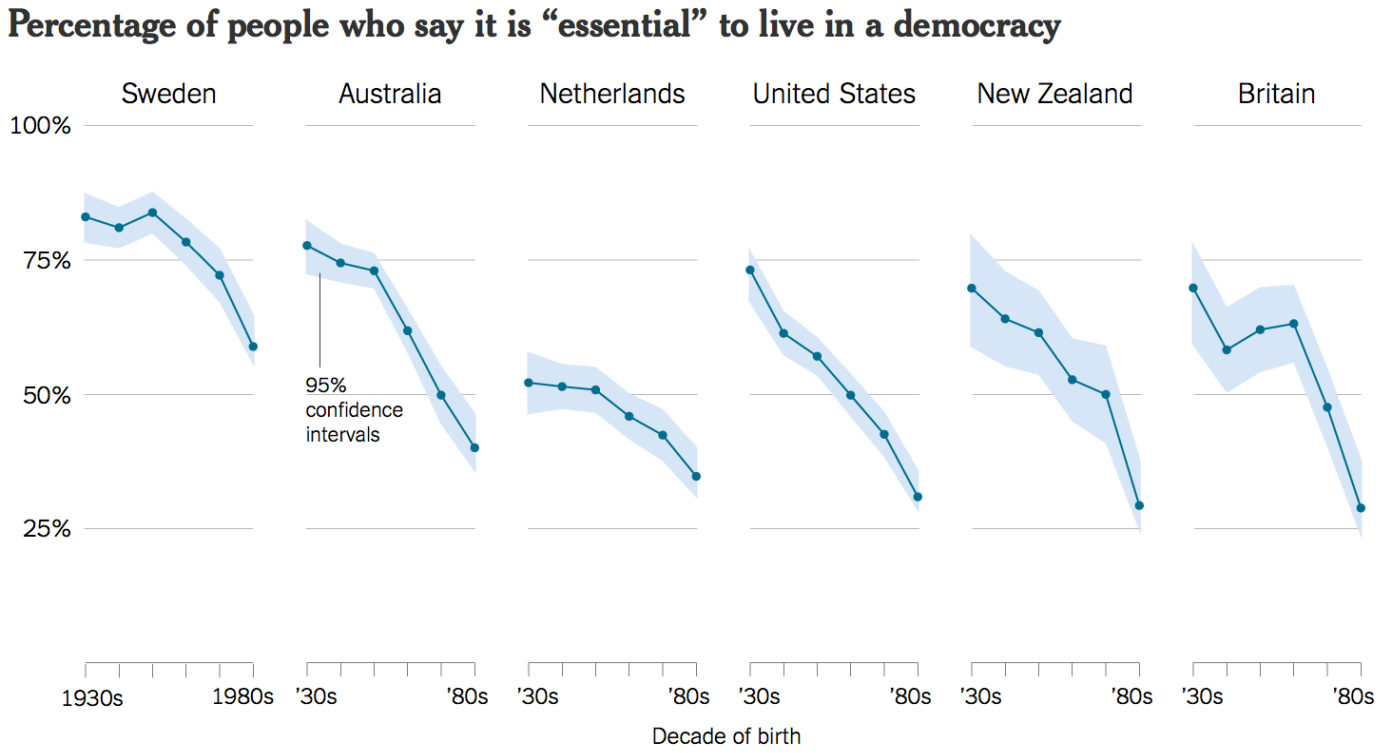


Source: Jackman, Robert M. 1980. “The Impact of Outliers on Income Inequality.” *American Sociological Review* 45(2): 344–347.

What makes bad figures bad? (from Healy 2019)

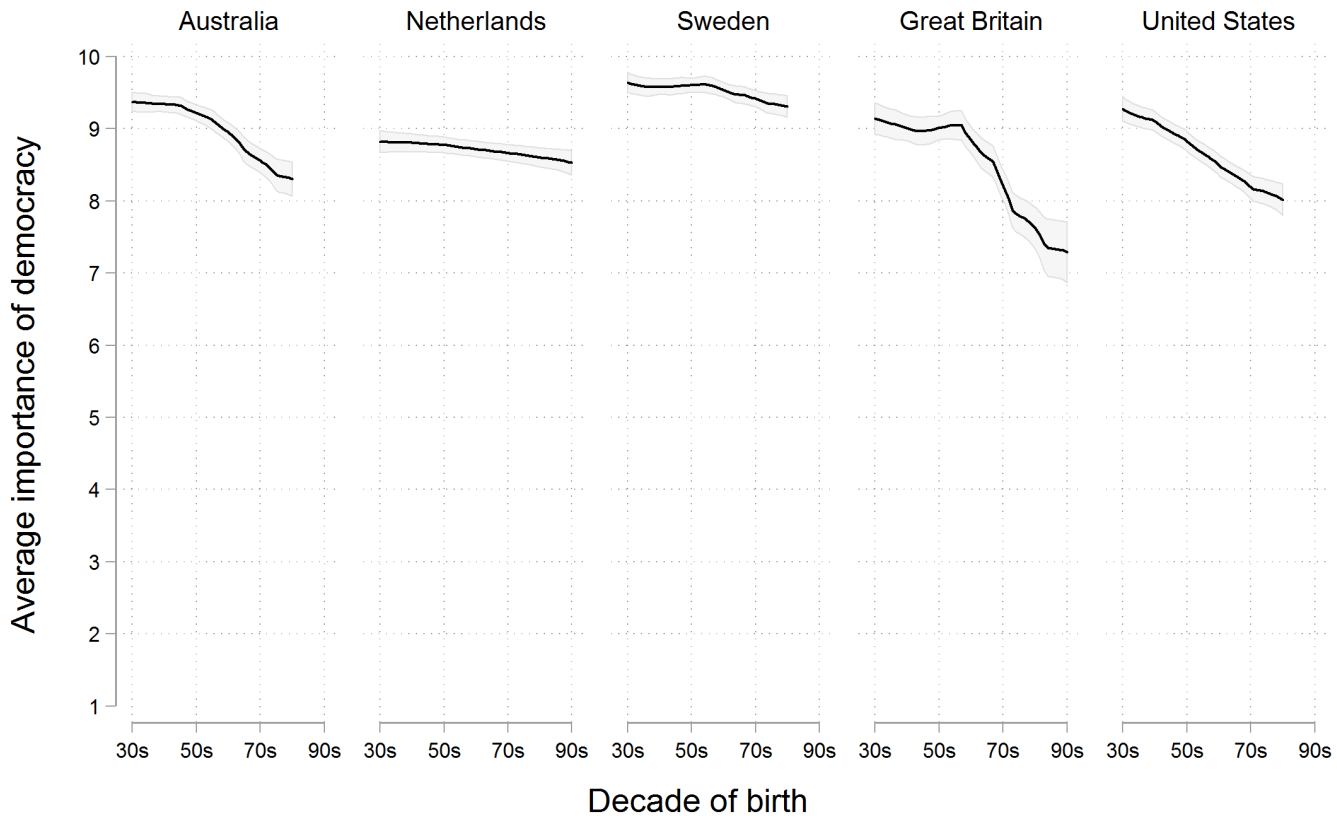


What makes bad figures bad? (from Healy 2019)



Source: Yascha Mounk and Roberto Stefan Foa, "The Signs of Democratic Deconsolidation," Journal of Democracy | By The New York Times

What makes bad figures bad? (from Healy 2019)



Graph by Erik Voeten, based on WVS 5

The caption is not an afterthought

The caption is what makes most figures self-sufficient

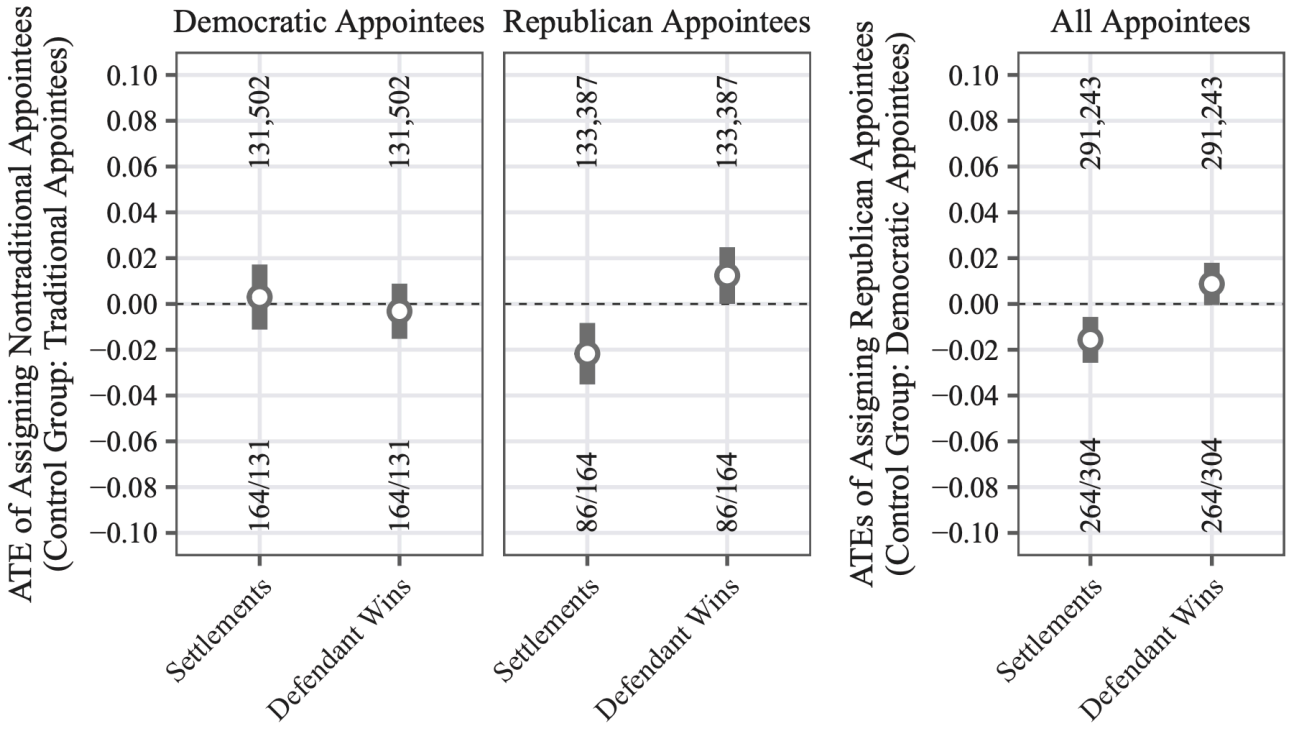
- Write it as a short paragraph, not a brief label
- It is often the only thing a reader of a paper/report reads

A caption should state important info, e.g.:

- **What is plotted**: the unit of observation, the variables, the estimator or transformation
- **Where it came from**: source, sample/coverage (N, years, geography), and any exclusions
- **How to read it**: what the bands, whiskers, dashed lines, shaded regions or reference lines mean
- **What to take away**: the finding the figure is evidence for

Copus, Hübert and Pellaton (2025)

FIGURE 4. Main Average Treatment Effects



Note: Each point plots an average treatment effect on a specific case outcome (depicted on the *x*-axis), along with a 95% confidence interval using judge-clustered standard errors. In the left two panels, we plot our main effects. In the right panel, we plot the average treatment effect of assigning Republican appointees to cases (instead of Democratic appointees), which we provide for comparison. For each estimate, we present the number of cases in the analysis (top number) and the number of treatment/control judges (bottom number). Full results for this plot are available in Table E.1 in Appendix E of the Supplementary Material.

Source: Copus, Ryan, Ryan Hübert, and Paige Pellaton. 2025. "Trading Diversity? Judicial Diversity and Case Outcomes in Federal Courts." *American Political Science Review* 119(2): 832–846. doi: 10.1017/S0003055424000625.

Outline

1. Some principles of data visualisation
2. Grammar of graphics and `plotnine`
3. Spatial data

The “grammar of graphics”

Wilkinson (2005): (statistical) graphics have a “grammar”

- That is: a set of mathematical and aesthetic rules for creating visual representations from data

The big (somewhat subtle) idea: data visualisation isn’t limited to a constrained set of pre-defined and formulaic “charts”

- The grammar allows us to innovate and create new kinds of visualisations

Python package `plotnine`: a layered version of Wilkinson’s grammar of graphics designed for use in Python

- Very powerful package for beautiful, bespoke visualisations
- Same syntax as R’s `{ggplot2}`

The “grammar” of `plotnine`

`plotnine` creates visualisations from data using **layers**

- A visualisation can have more than one layer
- Intuitively: creating a visualisation = stacking layers

Each layer contains:

- `data`: data to visualise
- `mapping`: links variables in data to visual properties
- `stat`: statistical transformations of data
- `geom`: controls the *type* of plotting object (line, point, etc)
- `position`: adjust overlapping objects

A running example: penguins

Best to see examples, like this nice one from R4DS:

```
from plotnine import *
from plotnine.data import penguins
print(penguins.head())
```

```
  species     island  bill_length_mm  ...  body_mass_g     sex  year
0  Adelie  Torgersen            39.1  ...       3750.0    male  2007
1  Adelie  Torgersen            39.5  ...       3800.0  female  2007
2  Adelie  Torgersen            40.3  ...       3250.0  female  2007
3  Adelie  Torgersen             NaN  ...          NaN     NaN  2007
4  Adelie  Torgersen            36.7  ...       3450.0  female  2007

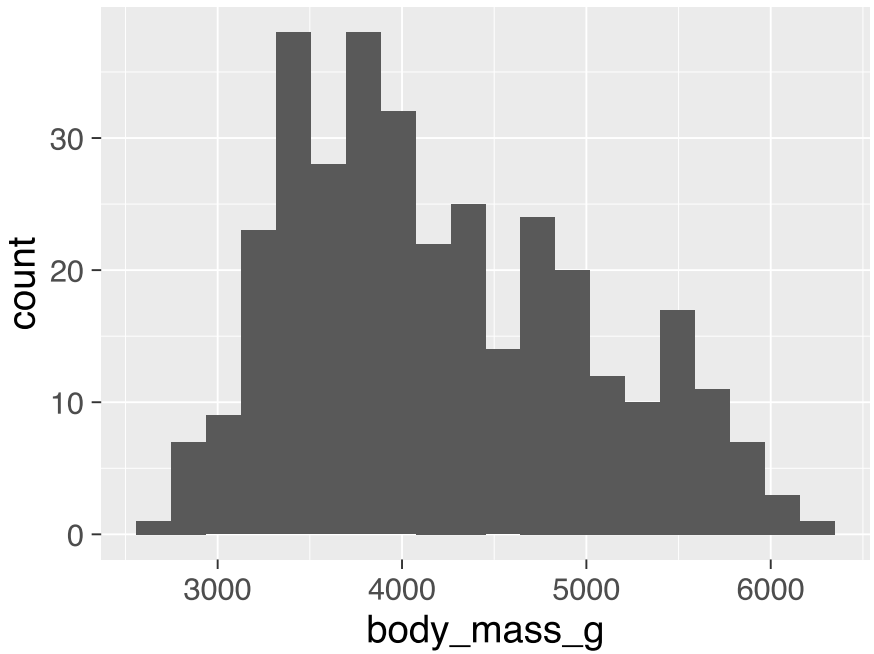
[5 rows x 8 columns]
```

Penguins (like humans!) vary in their weight

- How could we summarise this fact visually? **A histogram**

Penguins: univariate visualisation

```
z = ggplot()                                                    # initialise plot
z = (z +                                                        # add layers to `z`
      geom_histogram(data = penguins,                           # specify data
                     mapping = aes(x = "body_mass_g"),          # map variable to axes
                     stat = "bin",                              # histogram default
                     position = "stack",                        # histogram default
                     bins = 20))                                # histogram setting
z.show()                                                        # Show the plot
```



Penguins: bivariate visualisation

Now let's look at a bivariate relationship

- Flipper length and body mass

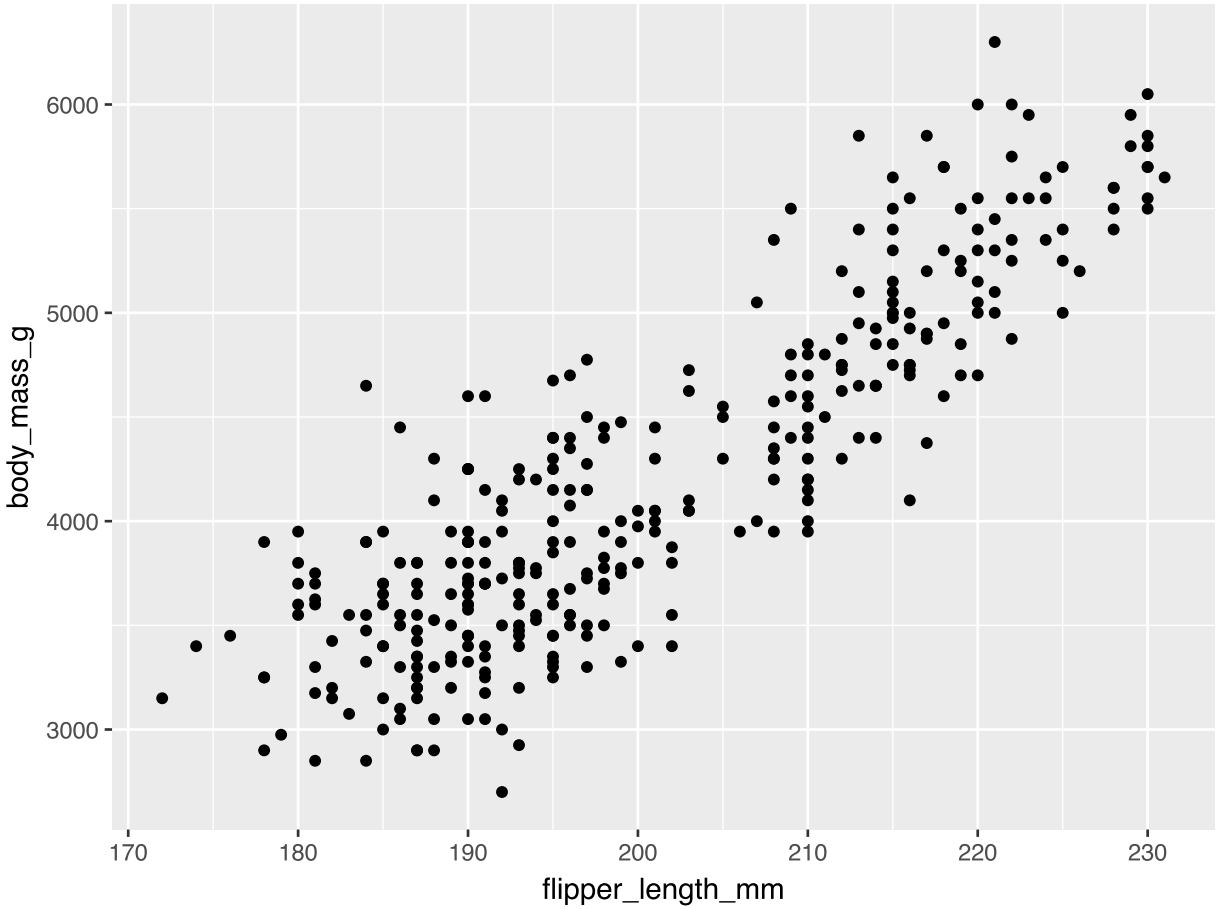
We can define `data` and `mapping` when we initialise

- All layers inherit these, unless otherwise specified

```
z = ggplot(data = penguins,                          # initialise with
           mapping = aes(x = "flipper_length_mm", # data & a mapping
                         y = "body_mass_g"))
```

Penguins: add a `geom` (points)

```
z = (z +
      geom_point())  # inherits the data and mapping
z.show()
```



More of the “grammar” of `plotnine`

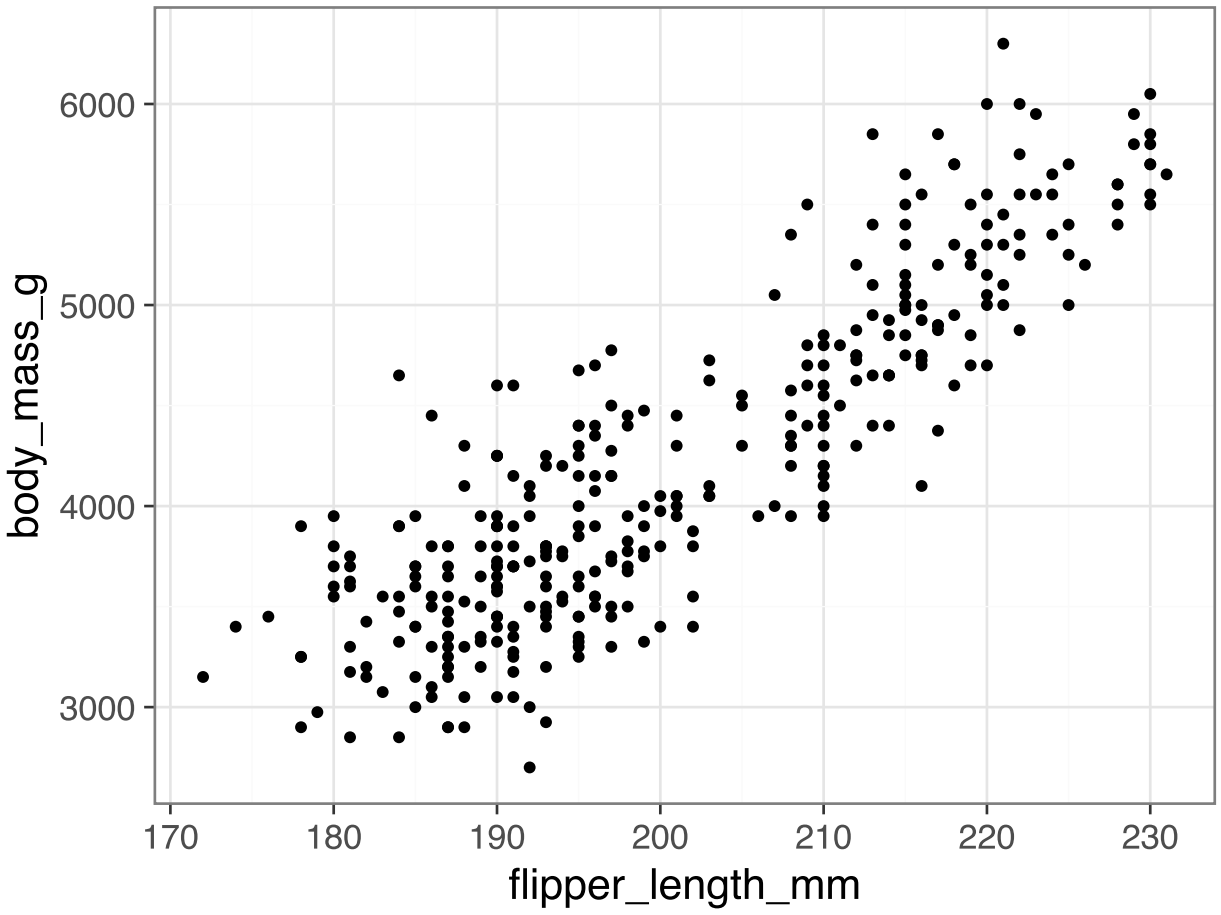
Layers are the most important component of the grammar, but there are four other major components

- **scales**: translation between variable ranges and graphical properties, e.g. linking values to colours and/or shapes
- **coordinates**: Coordinate system that, e.g. provides axes and gridlines
- **facets**: Breaking up the data into subsets, e.g. to be displayed independently on a grid
- **theme**: Parts that do not follow from the data, e.g. background colours, fonts, grid lines, legends etc.

Possibly a fifth: **metadata** (like labels/captions)

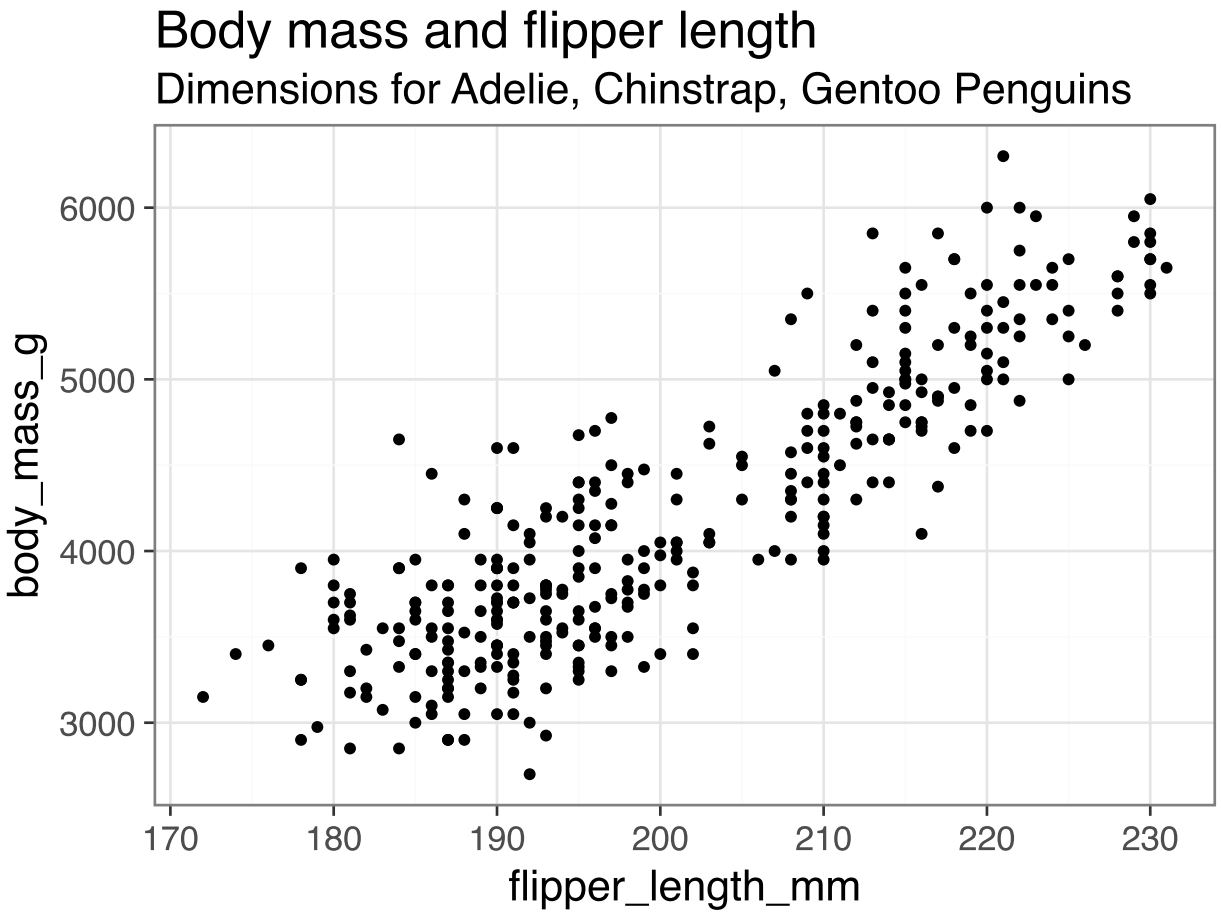
Penguins: adjust the (visual) theme

```
z = (z +
      theme_bw(base_size = 16)) # make bigger & remove grey bg
z.show()
```



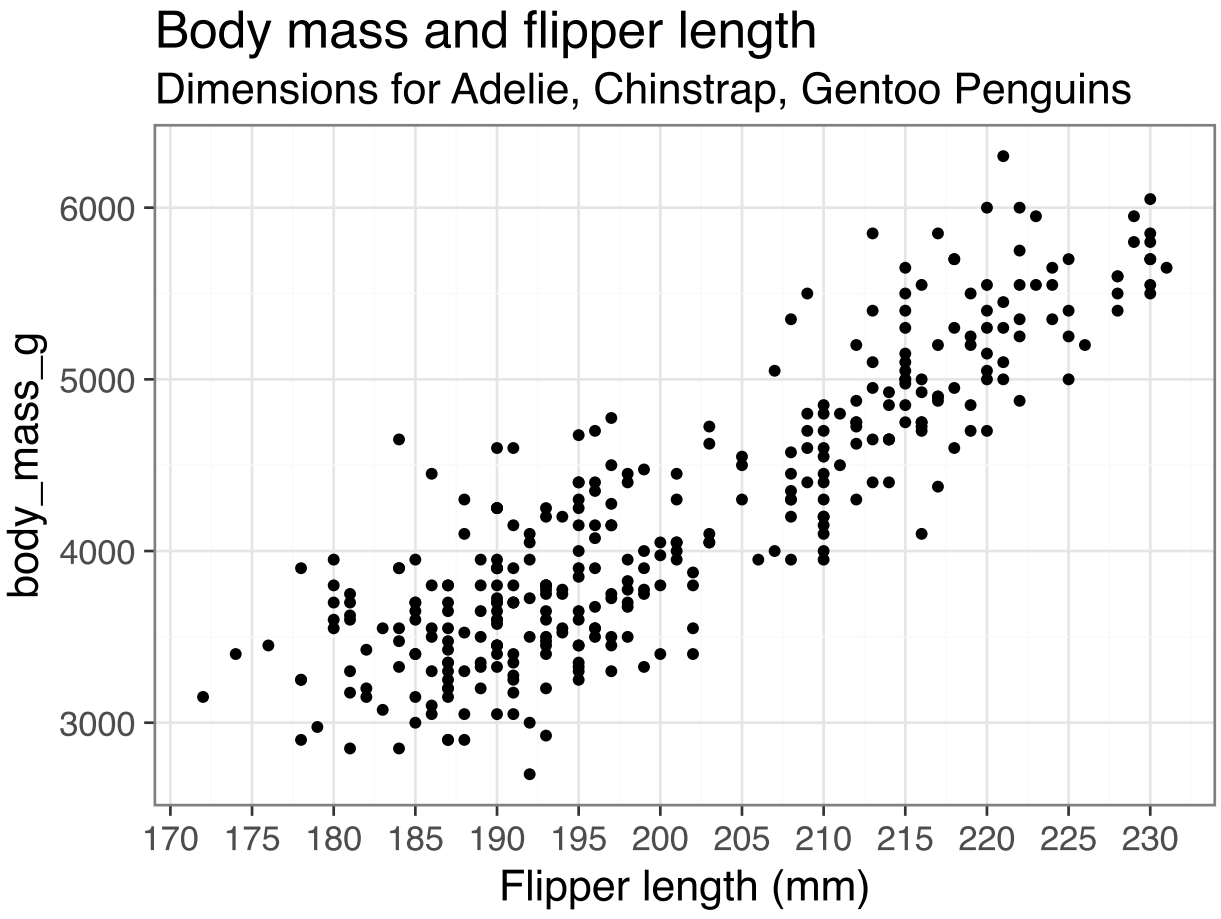
Penguins: add plot title

```
z = (z +
      labs(title = "Body mass and flipper length",
           subtitle = "Dimensions for Adelie, Chinstrap, Gentoo Penguins"))
z.show()
```



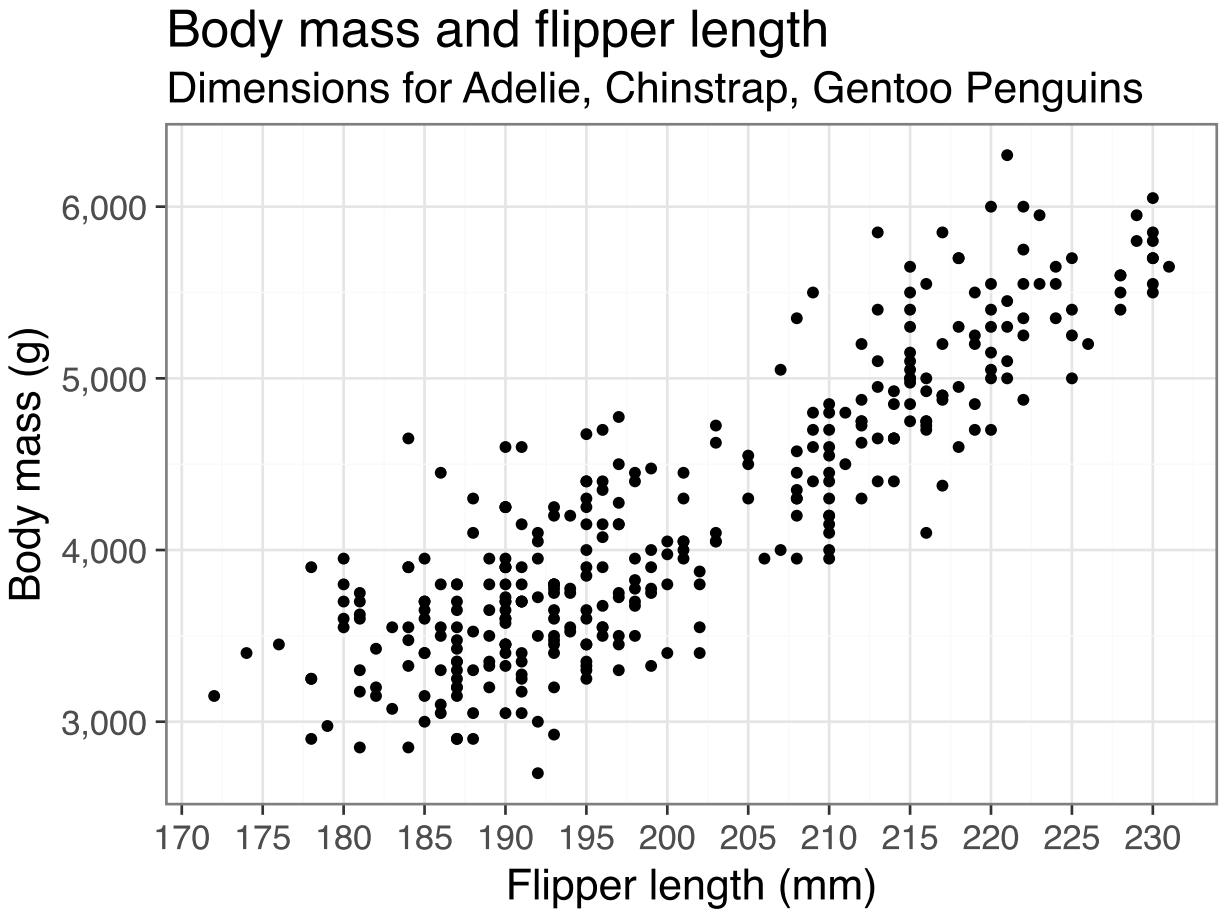
Penguins: modify x and y scales

```
z = (z +
      scale_x_continuous(name = "Flipper length (mm)",
                         breaks = range(170, 231, 5)))
z.show()
```



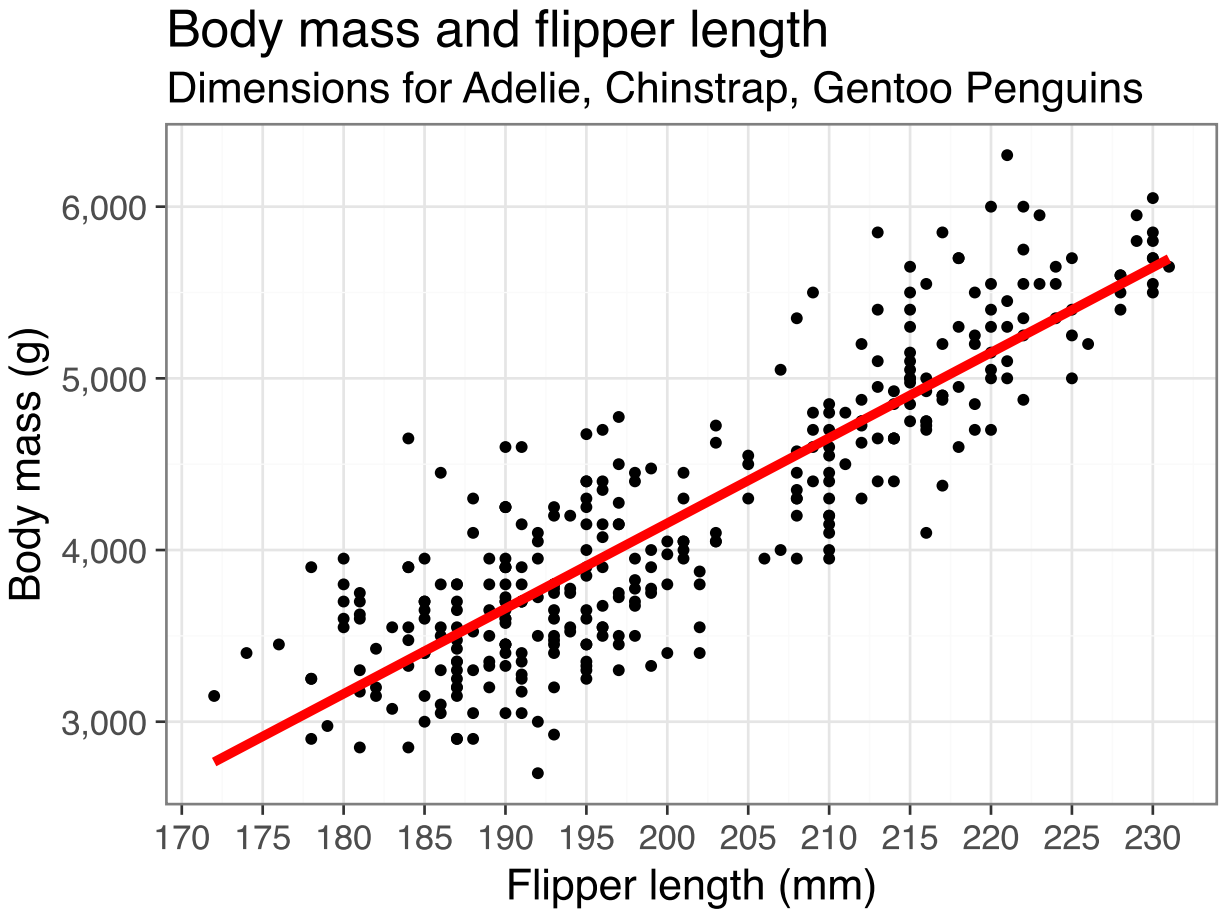
Penguins: modify x and y scales

```
from mizani.labels import label_comma
z = (z +
      scale_y_continuous(name = "Body mass (g)", labels = label_comma()))
z.show()
```



Penguins: overlay another layer

```
z = (z +
      stat_smooth(geom = "smooth", method = "lm",
                  se = False, size = 2, colour = "red"))
z.show()
```



Penguins: map `species` to colour/shape

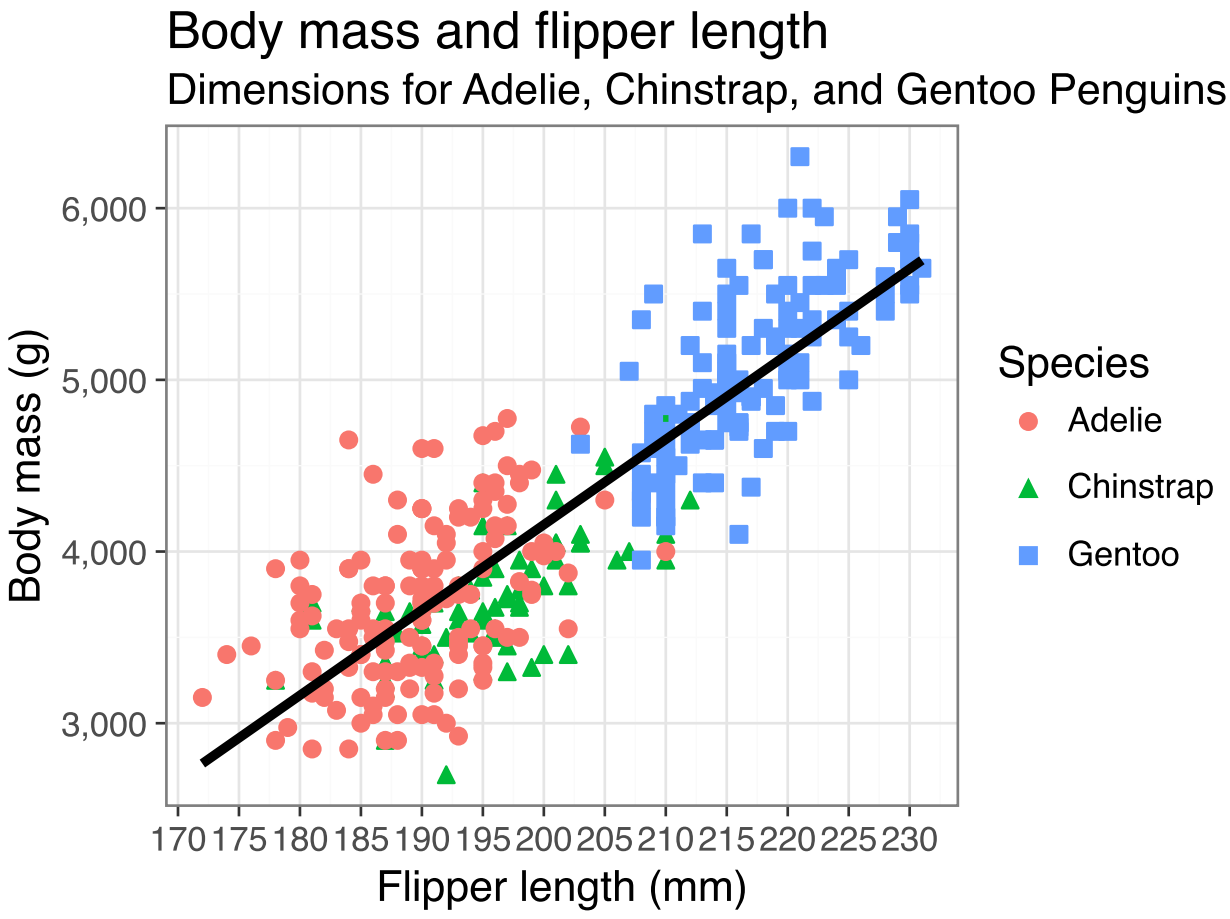
```
## Initialise a plot
z1 = ggplot(data = penguins,
            mapping = aes(x = "flipper_length_mm", y = "body_mass_g"))

## Add (large) points mapping species to colour/shape & regression line
z1 = (z1 +
      geom_point(mapping = aes(colour = "species", shape = "species"),
                 size = 3) +
      geom_smooth(method = "lm", se = False, size = 2, colour = "black"))

## Adjust aesthetic properties
z1 = (z1 +
      theme_bw(base_size = 16) +
      scale_x_continuous(name = "Flipper length (mm)",
                         breaks = range(170, 231, 5)) +
      scale_y_continuous(name = "Body mass (g)",
                         labels = label_comma()) +
      labs(title = "Body mass and flipper length",
           subtitle = "Dimensions for Adelie, Chinstrap, and Gentoo Penguins",
           colour = "Species", shape = "Species"))
```

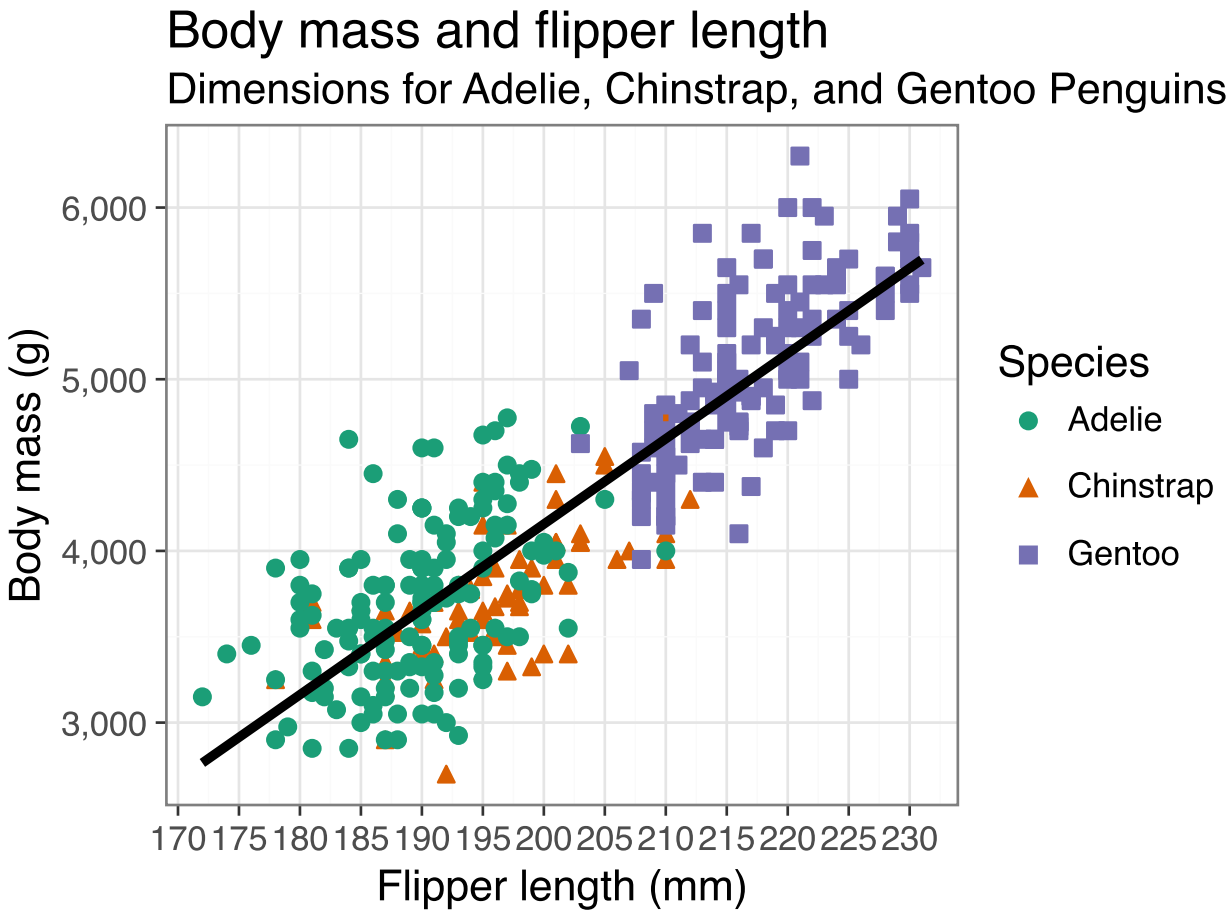
Penguins: map `species` to colour/shape

```
z1.show()
```



Penguins: modify colour scale

```
(z1 + scale_colour_brewer(type = "qual", palette = "Dark2")).show()
```



See: https://colorbrewer2.org/#type=qualitative&scheme=Dark2&n=3

Redundant scales

In the previous slide:

- Shape of dots adds no *new* information
- We call this **redundancy**
  - When two (or more) scales translate the *same* variable to different aesthetics
- Redundancy can overly complicate plots…
  - … but can also add clarity, improve accessibility

Facets and coordinates

Facets allow for multiple plots by mapping subsets of data

- E.g. separate scatterplots by island
- When you facet by a single variable we use a **wrap**
- When we facet by two (or more) variables, we use a **grid**

Coordinate systems "map the position of objects onto the plane of the plot" (Wickham 2010, p.13)

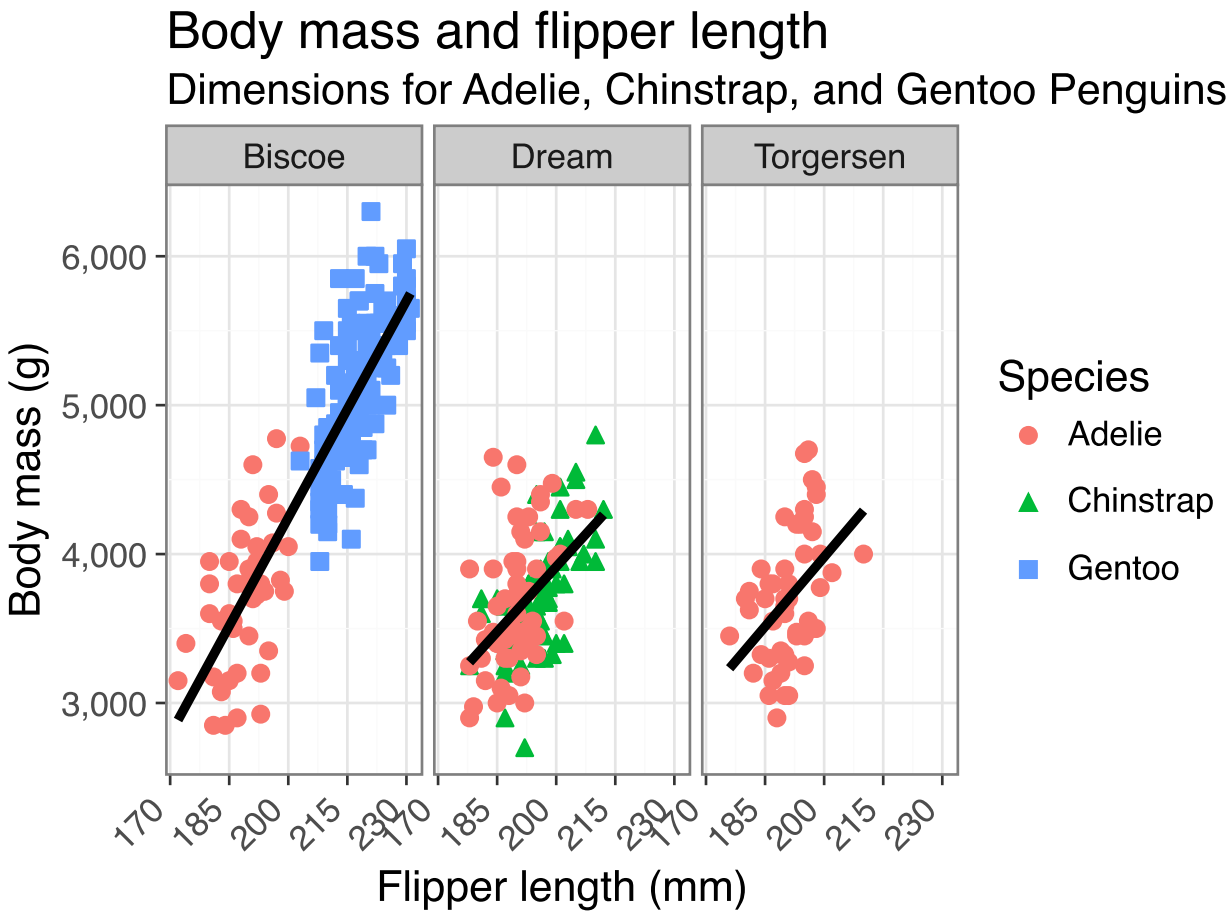
- In almost all cases we use **Cartesian coordinates**
  - Two orthogonal dimension (x, y)
- Alternative systems exist, like polar coordinates:
  - Allow you to draw circular distributions like pie-charts (eww!)

Penguins: create facets with `island`

```
z2 = (z1 +
        scale_x_continuous(name = "Flipper length (mm)",
                           breaks = range(170, 231, 15)) +
        theme(axis_text_x = element_text(rotation = 45, ha = "right")) +
        facet_wrap("island", nrow = 1))
```

Penguins: create facets with `island`

```
z2.show()
```



Outline

1. Some principles of data visualisation
2. Grammar of graphics and `plotnine`
3. Spatial data

What is spatial data?

Spatial data is data relating to physical space

Digitising space runs into a problem: space is continuous but digital data is discrete

Social science spatial data is mostly about Earth

- We will focus on 2D representations of Earth—maps

But spatial data raises two distinct issues:

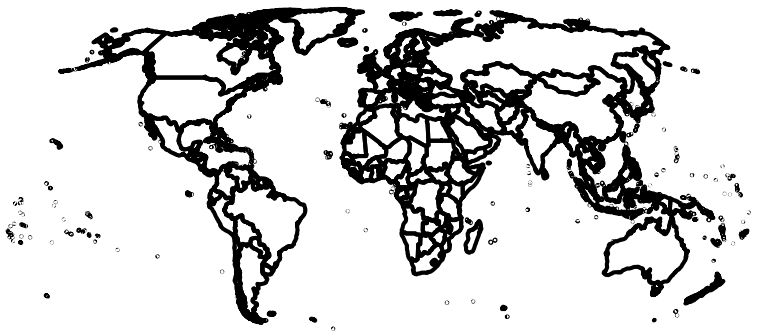
1. Earth may be continuous, but it's not *smooth* (bad for math)
   - Commit to a **datum**: a smooth 3D model of Earth
2. Space is 3D but maps are 2D
   - Commit to a **projection** that transforms the 3D model to 2D

Map projection

Mercator Projection
(without Antarctica)



Robinson Projection
(without Antarctica)



Coordinate reference systems

Coordinate reference system (CRS): a datum and a coordinate system that gives coordinates for real places

- **Geographic CRS** (no projection): coordinates are *angles* (longitude/latitude, in degrees)
- **Projected CRS**: coordinates are *distances* (x and y in metres) from an origin

There are trade-offs from different CRSs, e.g. projections distort some mix of directions, distances and areas

Two commonly used CRS:

- WGS 84: geographic. The datum behind GPS and the coordinates in most web applications
- British National Grid (BNG): projected, used in UK

Spatial data in Python

To make maps in Python, we'll use `geopandas`

- Defines spatial data with "simple features" that represent geometric data types + projection
- Spatial data is loaded into a `GeoDataFrame` with:
  - rows: geographic entities (countries, regions, etc.)
  - columns: relevant "metadata" about each geographic entity plus one column with **geometries** in a `geometry` column
  - `geometry` column contains vector data for plotting
- The coordinates used for plotting are "scaled" by the projection

Many ways to store vector data, but a commonly used one is "shapefiles" (`.shp`), which is what we will use

Reading shapefiles in Python

Read shapefiles into Python using `read_file()` from `geopandas`

```
import geopandas as gpd

# Read shapefile data into Python using geopandas
lf = gpd.read_file(os.path.join("data", "ESRI",
                                "London_Borough_Excluding_MHW.shp"))

# Peek at what the data looks like
print(lf[["NAME", "geometry"]].head())
```

```
                   NAME                                           geometry
0  Kingston upon Thames  POLYGON ((516401.6 160201.8, 516407.3 160210.5...
1               Croydon  POLYGON ((535009.2 159504.7, 535005.5 159502, ...
2               Bromley  POLYGON ((540373.6 157530.4, 540361.2 157551.9...
3             Hounslow  POLYGON ((521975.8 178100, 521967.7 178096.8, ...
4               Ealing  POLYGON ((510253.5 182881.6, 510249.9 182886, ...
```

```
# Which CRS is it in?
print(lf.crs.name)
```

```
OSGB36 / British National Grid
```

What is in the `geometry` column?

Each borough's shape is stored as a list of points

- Each point is an (x, y) pair in the data's CRS (here BNG, in metres)
- A **polygon** is the shape made by joining the points in order

```
westminster = lf.loc[lf["NAME"] == "Westminster", "geometry"].iloc[0]
print(westminster.geom_type)                    # type of shape
```

```
Polygon
```

```
print(len(westminster.exterior.coords))         # number of points
```

```
700
```

```
print(list(westminster.exterior.coords)[0:3])  # first three points
```

```
[(528549.5, 177903.8), (528542.6, 177949.4), (528540.1, 177958.4)]
```

Plotting spatial data

`geopandas` is just used to *load* spatial data

To plot, we use `plotnine` again

- `geom_map()` draws the shapes in the `geometry` column
- `coord_fixed()` keeps the map's proportions: one unit on x is the same length as one unit on y

Initialising a map

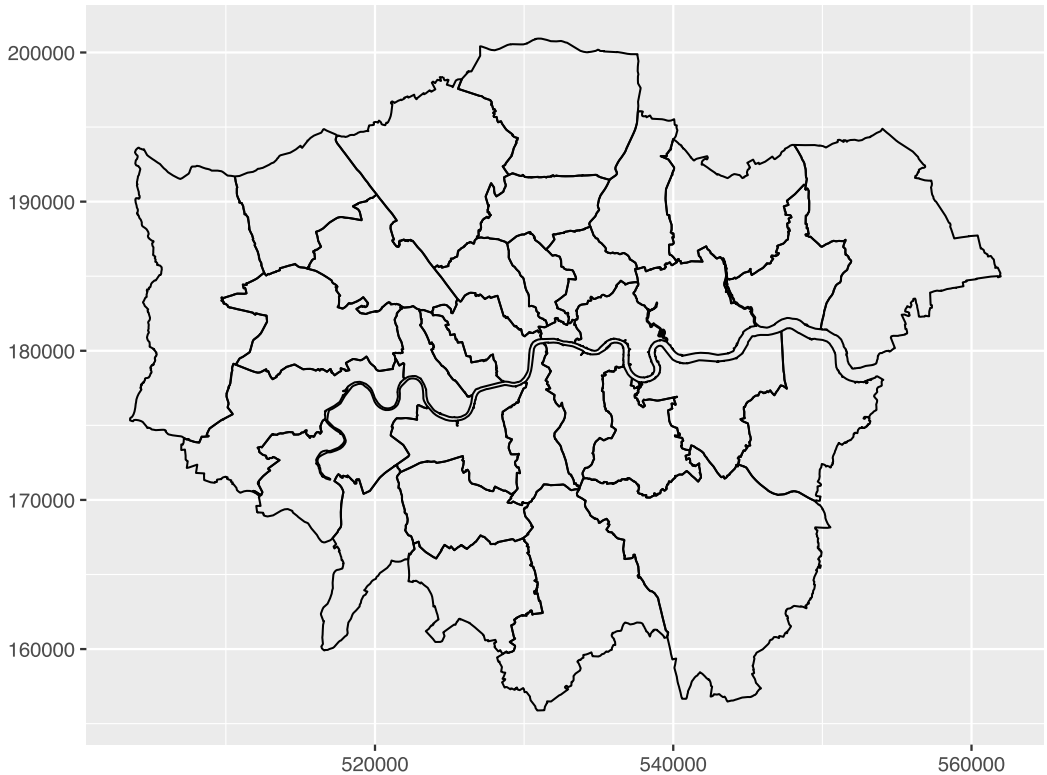
```
z = ggplot(data = lf)  # initialise a plot with the borough data
z.show()
```


Note:

- Nothing is in the plot yet (no mapping!)

Initialising a map

```
z = (z +
      geom_map(fill = "none",      # fill colour of plotted geographic units
               color = "black") + # border colour of plotted geographic units
      coord_fixed())               # keep the map's proportions
z.show()
```



Mapping in Python

You can add points to a map using latitude and longitude

- Find coordinates using Google Maps

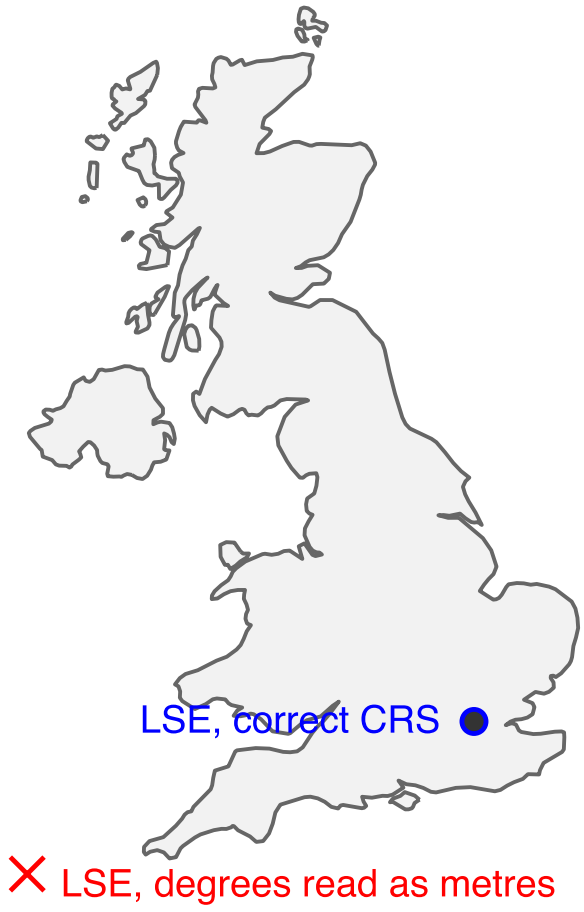
```
pf = gpd.GeoDataFrame({"name": ["LSE", "Heathrow"]},
      geometry = gpd.points_from_xy([-0.1164513, -0.4550471],
                                    [51.5144388, 51.4679903]),
      crs = 4326  # EPSG Code for WGS 84
     )
print(pf)
```

```
       name                    geometry
0       LSE  POINT (-0.11645 51.51444)
1  Heathrow  POINT (-0.45505 51.46799)
```

- Note, use EPSG code to identify your desired CRS

Mapping in Python

Same two coordinates $(-0.1165, 51.5144)$, with different CRS:



Mapping in Python

```
print(pf)             # original
```

```
       name                    geometry
0       LSE  POINT (-0.11645 51.51444)
1  Heathrow  POINT (-0.45505 51.46799)
```

```
pf = pf.to_crs(27700) # convert to the borough map's CRS (EPSG Code for BNG)
print(pf)
```

```
       name                        geometry
0       LSE  POINT (530796.147 181183.039)
1  Heathrow  POINT (507411.388 175467.327)
```

Mapping in Python

```
(ggplot() +
  geom_map(data = lf, fill = "#F2F2F2", color = "#B3B3B3", size = 0.5) +
  geom_map(data = pf, color = "red", size = 4) +
  geom_text(data = pf,
            mapping = aes(x = "geometry.x", y = "geometry.y", label = "name"),
            ha = "left", va = "bottom", size = 18,
            nudge_x = 600, nudge_y = 600) +
  coord_fixed() +
  theme_void()).show()
```



Joining data to a map

The borough shapefile has shapes, but no data about residents

- A separate file has panel data of births in each borough
- So we can merge the two tables on the GSS code

```
births = pd.read_csv(os.path.join("data",
           "births_by_mothers_country_of_birth_2001_to_2023.csv"))
births = births.loc[births["year"] == 2023, :]
lb = lf.merge(births, left_on = "GSS_CODE", right_on = "gss_code",
              how = "inner")
print(lb[["NAME", "total_births_all"]].head())
```

```
                   NAME  total_births_all
0  Kingston upon Thames              1607
1               Croydon              4939
2               Bromley              3408
3             Hounslow              3577
4                Ealing              4441
```

A choropleth map

A **choropleth** map fills each area with a colour that shows the value of a variable

- `aes(fill = "total_births_all")` maps a column to fill colour
- This is the same as `aes(colour = "species")` for the penguins

```
z = (ggplot(lb) +
      geom_map(mapping = aes(fill = "total_births_all"),
               color = "white", size = 0.2) +
      coord_fixed() +
      labs(fill = "Total births, 2023") +
      theme_void())
```

See: https://en.wikipedia.org/wiki/Choropleth_map

A choropleth map

